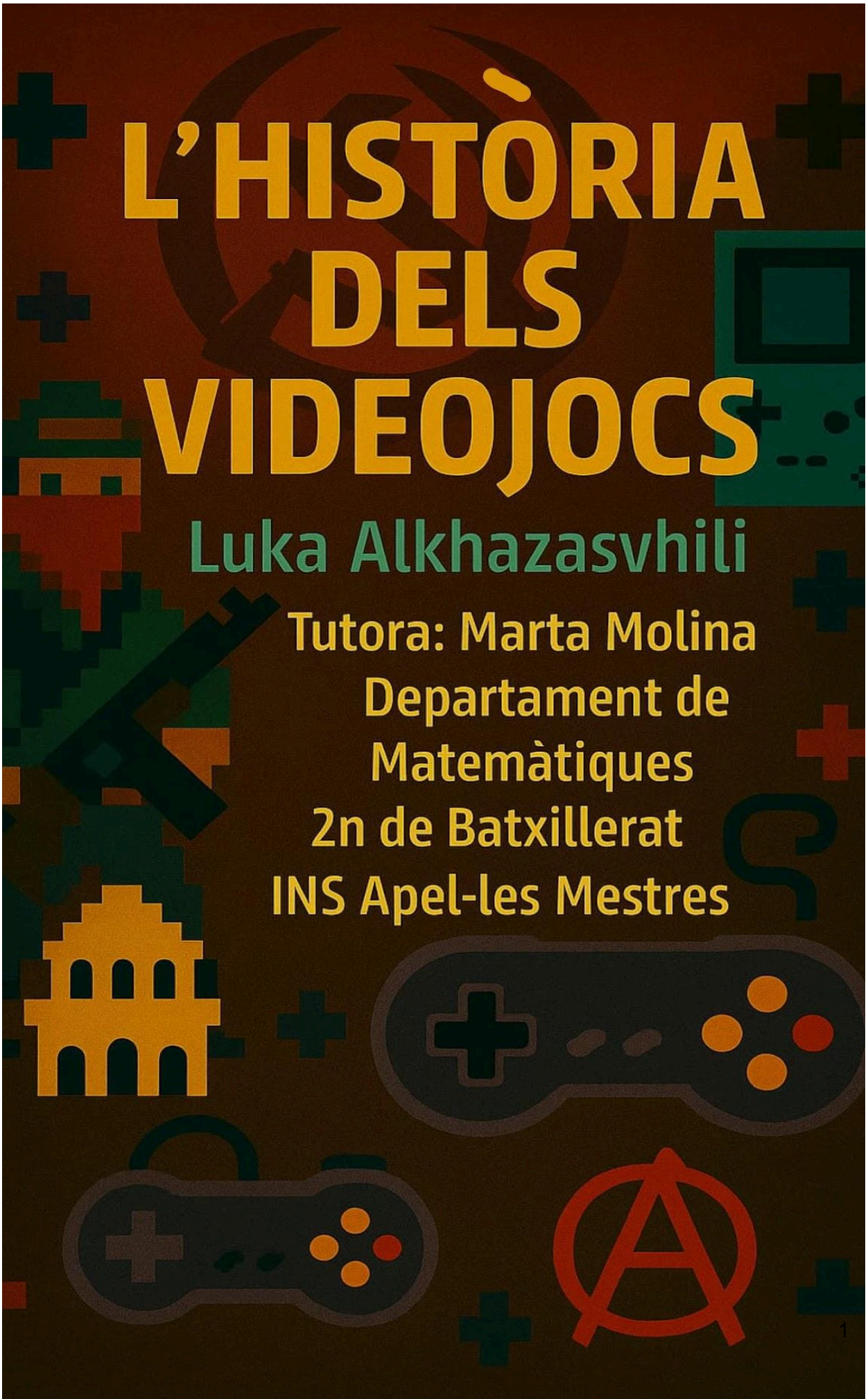

L'HISTÒRIA DELS VIDEOJOCS
Luka Alkhazasvhili
Tutora: Marta Molina
Departament de Matemàtiques
2n de Batxillerat
INS Apel-les Mestres

1. Justificació del treball **2**
2. Primer ordinador **3**
2.1 John Mauchly 3
2.2 John Presper Eckert 4
3. Els primers videojocs **5**
3.1 OXO(1952) 5
3.2 Tennis for Two(1958) 5
3.3 Spacewar(1962) 6
4. Primeres consoles **7**
4.1 “Serie Odyssey” **7**
4.2 Crisis dels videojocs 7
5. Segona generació (4 bit) **8**
5.1 Magnavox Odyssey 8
5.2 Atari 2600 i Atari 5200 8
6. Procés de creació d’un videojoc **10**
6. 1 Tipus de videojocs 10
6.1.1 Aventura 10
6.1.2 Esports 11
6.1.3 Estratègia 11
6.1.4 Clickers 13
7. Motors per programar un videojoc **14**
7.1 Unreal Engine 5 14
7.2 Unity 3D 14
7.3 Game Maker 14
7.4 Lumber Yard 14
8. Llenguatges de programació **15**
9. La meva part pràctica **15**
10.Creació del meu propi videojoc **16**
12. Conclusions **35**
11. Webgrafia **36**

1. Justificació del treball

He triat aquest tema de *TR* perquè des de ben petit m'ha agradat molt el món dels videojocs i volia aprofundir més en el meu coneixement de programació ficant-me un repte de crear el meu propi videojoc.
La meva hipòtesi és que amb els meus coneixements previs dels llenguatges de programació C i Python, serè capaç de crear un videojoc funcional des de zero i aprendre a usar un altre llenguatge de programació

2. Primer ordinador

L'ENIAC (*Electronic Numerical Integrator and Computer*), també conegut com el "Projecte PX", va ser construït entre els anys 1943 i 1945 per John Mauchly i John Presper Eckert, juntament amb enginyers com Robert F. Shaw, Chuan Chu, Thomas Kite Sharpless, Arthur Burks, Harry Huskey, Iredell Eachus Jr i Jack Davis. Aquest ordinador és considerat com el primer ordinador d'ús general de la història. L'ordinador ocupava una habitació d'aproximadament 167 m² i pesava 30 tones. Estava format per molts components, entre ells:

Tubs de buit

Els tubs de buit es feien servir en aquell moment com a transistors, però eren molt poc eficients, ja que tenien una durada molt curta i s'havien de substituir sovint. Es va descobrir que es feien malbé més ràpidament si s'apagaven i encenien contínuament, per la qual cosa es va optar per mantenir l'ENIAC encès permanentment, cosa que permet que un mateix tub durés aproximadament dos dies.

2.1 John Mauchly



John William Mauchly va néixer el 30 d'agost de 1907 a Cincinnati,

Ohio, i va créixer a Chevy Chase, Maryland.

Va morir el 8 de gener de 1980.

Va obtenir la Beca d'Enginyeria de l'Estat de Maryland,

la qual li va permetre ingressar a la "Johns Hopkins University"

L'any 1925 com a estudiant de grau en el programa d'Enginyeria Elèctrica.

El 1927 va passar directament a un programa de doctorat i va ser transferit al programa de postgrau en Física de la mateixa universitat. Va completar el doctorat el 1932 i posteriorment es va convertir en professor de física a l'Ursinus College", a prop de Filadèlfia, on va ensenyar entre 1933 i 1941.

El 1941, el Dr. Mauchly va fer un curs d'electrònica militar a la "Moore School of Engineering", pertanyent a la "University of Pennsylvania". Allà va conèixer John Presper Eckert, un graduat recent d'aquesta escola. Mauchly va acceptar un lloc com a professor a la Moore School, un centre especialitzat en computació militar.

Eckert el va animar a creure que els tubs de buit es podien fer més fiables mitjançant pràctiques d'enginyeria adequades. El principal problema d'estudi a la Moore School era la balística, específicament el càlcul de taules de tir per a les noves armes de la marina dels EUA. A començaments dels anys 40, Mauchly va escriure un memoràndum titulat: *"Ús de tubs de buit d'alta velocitat per realitzar càlculs."*

2.2 John Presper Eckert

John Adam Presper Eckert Jr. va néixer el 9 d'abril de 1919 a Filadèlfia, EUA, i va morir el 3 de juny de 1995 als 76 anys.



Des de petit va demostrar una gran habilitat per a les matemàtiques, amb una mentalitat d'inventor. El 1937 va ingressar a la *University of Pensilvània* per estudiar Enginyeria Elèctrica, graduant-se l'any 1941.

Després de graduar-se, va continuar a la universitat com a instructor en cursos d'electrònica i es va dedicar a estudis sobre programació. Va ser en aquests cursos on va conèixer John William Mauchly, amb qui va establir una amistat que duraria tota la vida.

3. Els primers videojocs

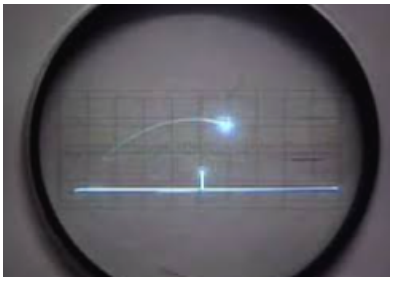
3.1 OXO(1952)

Un dels primers videojocs fou un joc anomenat *OXO*, conegut actualment com el “tres en ratlla”. Dissenyat pel professor Alexander S. Douglas el 1952 com a part de la seva tesi sobre la interacció entre humans i ordinadors a la Universitat de Cambridge, utilitzava un marcador de telèfon giratori per controlar el joc.



3.2 Tennis for Two(1958)

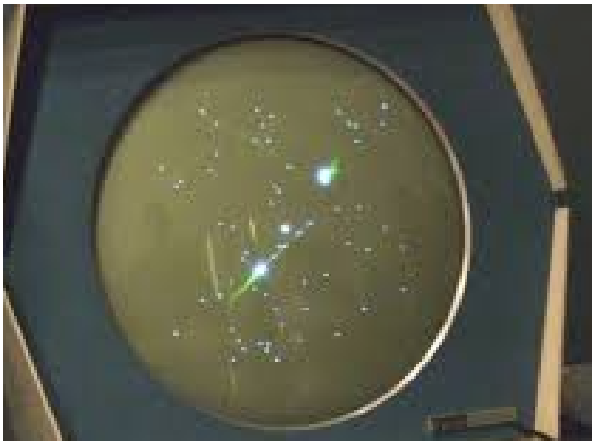
Creat per Steve Russell amb la col·laboració de Martin Graetz, Wayne Wuutanen, Bob Saunders, Steve Piner i altres, es va programar per al miniordinador DEC PDP-1. El joc consistia en dues naus espacials, anomenades “l’agulla” i “la falca”, controlades per dos jugadors que es combatien mentre orbitaven una estrella amb gravetat. Disposaven d’una quantitat limitada d’armament i combustible, i estaven en moviment constant, fins i tot sense aplicar acceleració.





3.3 Spacewar(1962)

Creat per Steve Russell en col·laboració amb Martín Graetz, Wayne Wuutanen, Bob Saunders, Steve Piner i altres, va ser programat per al miniordinador DEC PDP-1.



El joc consisteix en 2 naus espacials anomenades l'agulla i la falca controlades per jugadors que s'enfronten en una batalla mentre naveguen al voltant del pou gravitatori d'una estrella tenint una quantitat limitada d'armament i combustible per maniobrar, trobant-se en moviment continu. Fins i tot si el jugador no aplica cap acceleració.

4. Primeres consoles

Per parlar de la història de les consoles, cal començar amb la generació dels 1-bit, que fa referència als videojocs i consoles disponibles entre 1972 i 1983. Algunes de les consoles més destacades d'aquesta època són:

4.1 "Serie Odyssey"

Atari Home Pong



Coleco Telstar



Color TV-Game



Tot i que la generació va finalitzar amb *Color TV-Game* l'any 1980, molts fabricants ja havien abandonat el mercat una dècada abans, arran de la crisi dels videojocs del 1977.

4.2 Crisis dels videojocs

Aquesta crisi es va produir principalment per la saturació del mercat de consoles i videojocs. Hi havia moltes empreses intentant capitalitzar la demanda, la qual cosa va portar a una oferta de productes de qualitat molt baixa. Això va fer que els consumidors perdessin la confiança i les vendes baixessin dràsticament, passant de 3.000 milions de dòlars a només 100 milions en tan sols un any.

5. Segona generació (4 bit)

Les consoles de segona generació van aparèixer entre 1976 i 1992. La majoria de jocs d'aquesta època estaven integrats dins la consola, a diferència de generacions posteriors on s'utilitzaven discos o cartutxos extraïbles.

Les capacitats gràfiques eren molt limitades: punts, línies o blocs senzills; alguns models només mostraven dos colors i el so era molt bàsic o inexistent.

5.1 Magnavox Odyssey

Odyssey 2:



Llançada el 1978, a finals de l'era 1-bit, aquesta consola basada en microprocessadors es va distribuir per Philips a Europa. Va millorar molt en efectes de so i música, convertint-se en la tercera més venuda de la seva generació, amb més de 2 milions d'unitats fins al 1982, quan va ser discontinuada.

5.2 Atari 2600 i Atari 5200

L'any 1977, Atari (fundada el 1972) va llançar la *Video Computer System (VCS)*, més coneguda després com Atari 2600. Aquesta consola aprofitava els drets exclusius d'alguns dels jocs d'arcade més populars per compensar la seva tecnologia ja antiga. Això, combinat amb un preu competitiu, la va convertir en un èxit de vendes, superant els 30 milions d'unitats.

Més tard, Atari va llançar l'*Atari 5200* per competir amb *Intellivision*. Malgrat les seves millores tècniques, les baixes vendes i l'escassetat de jocs van fer que fos retirada del mercat dos anys després del seu llançament.

6. Procés de creació d’un videojoc

6. 1 Tipus de videojocs

Quan estem parlant dels tipus de videojocs, aquests es classifiquen depenent del contingut del videojoc, com ara els jocs de

6.1.1 Aventura

Els videojocs d'aventura solen ser aquells que et fan enfrontar-te a enemics ia diferents nivells, durant el període d'exploració hauràs de buscar i trobar diferents objectes, de vegades fins i tot mitjançant missions, aquest tipus de videojocs solen tenir un bon fil d'història i una durada moderada.
Alguns dels exemples més coneguts:
a. Tomb Raider



Un joc que segueix l'aventura d'una arqueòloga (Lara Croft), el joc consisteix que el protagonista es fica en tombes i ruïnes antigues a la recerca de tresors i artefactes llegendaris, en el seu camí ha d'esquivar trampes, enfrontar-se a enemics i resoldre endevinalles.

b. The witchers 3

Un joc on el protagonista Geralt de Rivia, un caçador de monstres, viatja per un món enorme ple d'aventures a la recerca de Ciri, una noia amb poders increïbles que bàsicament és com la seva filla adoptiva, i durant el camí s'enfronta a un munt d'embolics entre ells: animals màgics, reis i guerres.



6.1.2 Esports

Doncs com bé diu el nom aquests jocs solen ser les recreacions d'esports reals, aquests jocs solen requerir més habilitat, per la qual cosa això significa que s'hi pot millorar a mitjà de la pràctica

Alguns dels exemples més coneguts:



a. Fútbol



b. Basketball

6.1.3 Estratègia

Aquests jocs, com bé diu el nom, solen ser jocs on has de pensar en una estratègia per vèncer l'enemic, aquest tipus de jocs no tots els agrada perquè té les seves dificultats atès que no sempre s'enganxa l'estratègia a la primera i de vegades necessita dedicar-li un temps per entendre com funciona el joc.

Alguns dels exemples més coneguts:



a. Cities Skylines:

Un joc on et converteixes en l'alcalde d'una ciutat i la construeixes des de zero, dissenyant les carreteres, zones residencials, comercials, industrials, gestionar serveis com l'electricitat, l'aigua i el transport públic, enfrontant-te a desafiaments com el trànsit la contaminació i el pressupost,



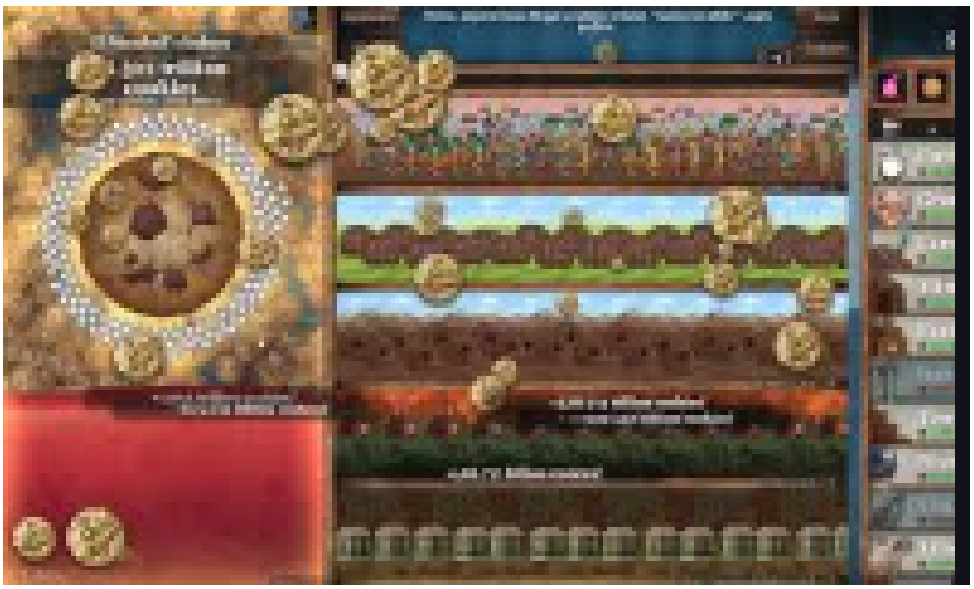
b. League of Legends:

League of Legends o també conegut com (LOL) és un joc multijugador d'estratègia que bàsicament consta en 2 equips de 5 persones que competeixen per destruir la base enemiga (el Nexo) cada jugador pot triar un campió únic amb habilitats úniques, l'objectiu és utilitzar el treball en equip per derrotar enemics i controlar àrees estratègiques.

Aquestes classes de jocs solen ser els més comuns, encara que n'hi ha més, una altra cosa que també sol passar és que un videojoc sol tenir més d'una de les franges dites anteriorment.

6.1.4 Clickers

Aquest tipus de jocs s'estan tornant molt famosos últimament, en aquests jocs el que has de fer és fer clics repetidament a un objecte per guanyar punts o recursos, usant aquests recursos per comprar millores.
Alguns dels exemples més coneguts:



a. Cookie Clicker:

Un exemple clau d'aquesta categoria, “Cookie Clicker” és un joc en què comences fent clic a una galeta per guanyar-ne d'altres. Amb aquestes galetes pots comprar millores i personatges que fan galetes per tu. Com més jugues, més ràpid aconsegueixes galetes, i tot es torna cada cop més boig i addictiu.

7. Motors per programar un videojoc

7.1 Unreal Engine 5

Dels motors més famosos actualment és el Unreal Engine 5, aquest motor va ser creat per Epic Games, una companyia molt gran i coneguda pel desenvolupament de videojocs com Fortnite, Unreal Engine 5 és dels més potents ara mateix on pots arribar a fer un videojoc en 3D des de zero, també és molt conegut per les seves capacitats de poder arribar a un realisme molt alt

7.2 Unity 3D

Unity, igual que Unreal Engine, és molt conegut amb una comunitat de desenvolupadors bastant gran, però alguna cosa que destaca més és que és molt més simple d'aprendre a usar i també pot ser usat en entorns 2D

7.3 Game Maker

Un motor de desenvolupament de videojocs, però en aquest cas per a 2D,alguna cosa que m'ha agradat molt d'aquest programa és que té una quantitat de tutorials molt extensos, d'aquesta manera pots programar un videojoc sense tenir molts coneixements, amb això vull dir que és molt amistós amb les persones que han començat recentment a programar

7.4 Lumber Yard

Aquest motor creat per Amazon, sí, l'has llegit bé, Amazon, és molt curiós atès que incorpora una cosa nova que uns altres no tenen que és la compatibilitat amb la plataforma de streaming Twitch, això permet als espectadors interactuar amb el joc mentre el jugador està jugant mitjançant l'execució de comandos.

8. Llenguatges de programació

El més essencial per a programar és el llenguatge de programació, n'hi ha molts, cadascun amb els seus avantatges i desavantatges, però el que tenen en comú és que ens permet als humans traduir des d'un idioma més semblant a la qual solem usar en el nostre dia quotidià a llenguatge de màquina que solen ser 0 i 1

Diferents Llenguatges
C, C++, C#
Java
JavaScript
Python
Lua

Una cosa que hauria de ser esmentat és que molts llenguatges no estan en aquesta llista perquè no solen ser usats globalment, però solen tenir moltes coses en comú en l'àmbit de funcionament.

9. La meva part pràctica

Després d'haver vist els diferents tipus de jocs de llenguatges i motors que hi ha, ara el que hem de fer és fer servir una mica la nostra creativitat, pensar en com volem que sigui el nostre videojoc, els personatges, on està passant l'acció. Ara només falta posar-se a la feina per a fer venir les nostres idees realitat.

10.Creació del meu propi videojoc

Com a part pràctica del TR he decidit fer el meu propi videojoc en 2D, a l'inici tenia planejat un joc que era com un “escape room”, bàsicament constava a anar en contra de les normes del joc, però després el vaig pensar i vaig arribar a la conclusió que seria difícil de fer-ho i que es faria molt avorrit. Llavors vaig decidir fer un joc tipus” CLICKER “on has d'anar matant monstres, per a aconseguir monedes i millorar el teu personatge per a així fer-te més poderós i poder derrotar-los més fàcilment.

El primer que he fet és triar Game Maker per a fer el meu projecte, atès que el volia fer en 2D i és molt fàcil d'usar comparat amb els altres per la facilitat que tinc per a trobar molts tutorials. A més el format de l'entorn de programació m'ha agradat més que la resta perquè em resulta més còmoda.

La primera part del procés va ser explorar una mica com funciona el programa en si, investigat el que són per exemple: els sprites.

Els sprites bàsicament són la part visual d'un objecte dins del joc. Podria ser un personatge un fons o un monstre, la interfície que té Game Maker per a la creació de sprites és bastant simple i amb una mica d'investigació pots arribar a dominar el seu ús.

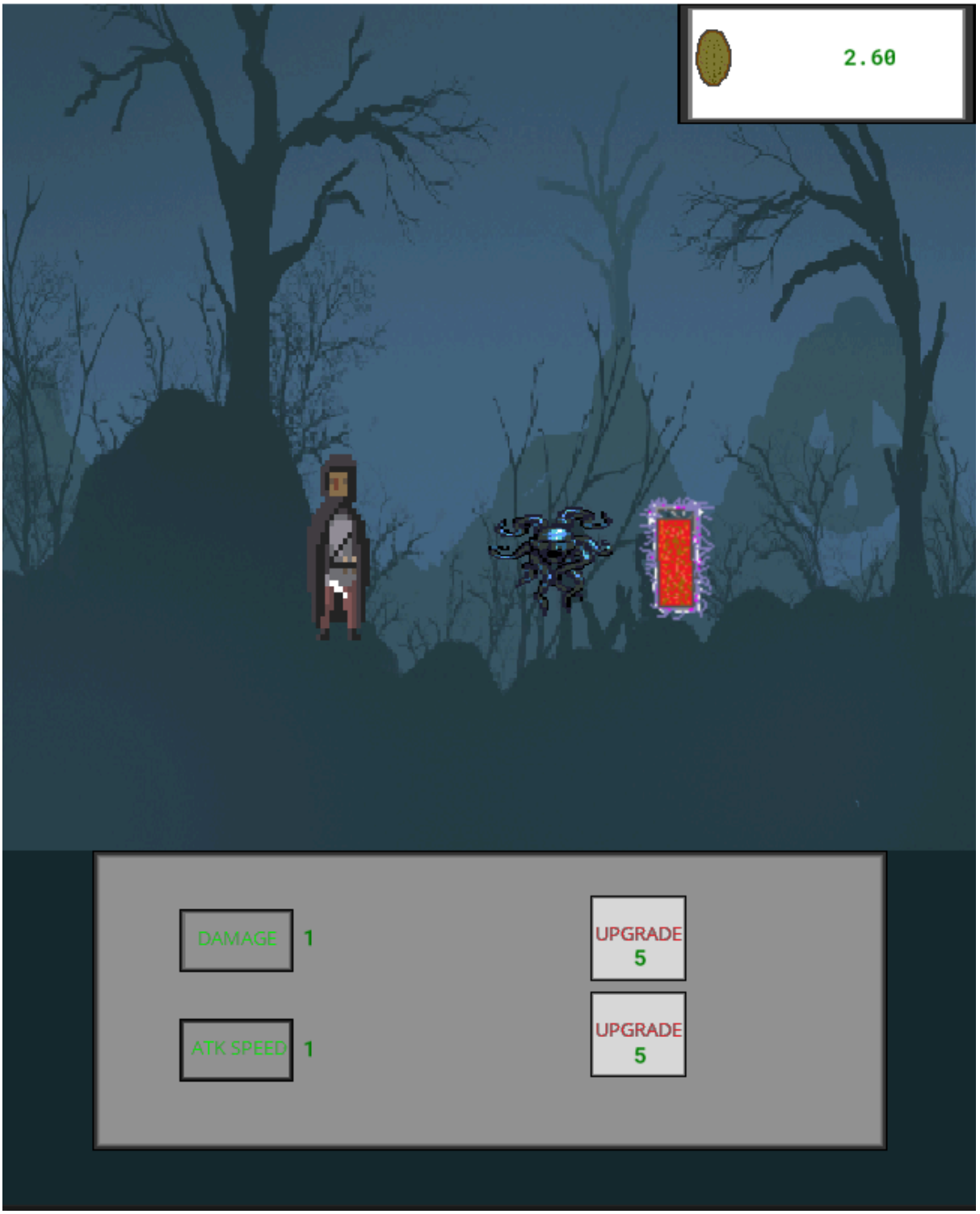
Després d'entendre més o menys com funcionaven els sprites em vaig posar a mirar com seria la programació en Game Maker, això és important perquè pot ser que el programa utilitzi un idioma de programació que per a ti sigui desconegut, la qual cosa va ser el meu cas, això podria haver-me dificultat molt més el procés de la creació/aprenentatge. Però el llenguatge de programació de Game Maker anomenat “GML CODE” i per sort era molt semblant a c+, amb la qual cosa ja tenia una mica més d'experiència gràcies a l'optativa de programació durant el primer any de batxillerat.

El primer que vaig fer abans de començar a programar el meu propi des de zero va ser mirar ja un joc fet per una altra persona, aquest programa tenia algunes funcions molt similars a la meva idea.

Després d'haver-me descarregat la plantilla em vaig posar a obrir els arxius a mirar totes les coses possibles, els objectes, sprites, el codi, literalment tot.

Després d'haver mirat tot el que tenia el programa en la meva ment va haver-hi la idea de “intentaré afegir algunes coses haver-hi com va”, ja us puc dir que després de 2 minuts el programa ja no funcionava. Això va ser per diverses raons una d'elles és que el programa no estava fet per mi, per la qual cosa no tenia ni idea de com funcionava ni d’on estava establert cada cosa, d'altra banda, va ser el simple fet que literalment feia 3 hores que mirava com funcionava el programa llavors tampoc tenia molta experiència.

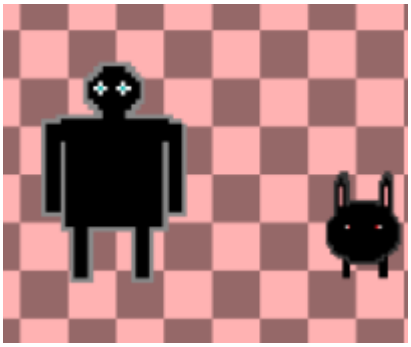
Després d'aquest primer fracàs vaig dir, “faré el meu propi des de zero” exactament, la meva decisió va ser enfrontar-me a alguna cosa que no havia fet mai tenint bàsicament 5% de coneixement requerit, la veritat no mentiré a l'inici va ser molt difícil per raons com no saber on has de crear cada objecte o sprite, o no saber com funcionen algunes funcions que venien ja directament amb el Game Maker, però després d'estar a poc a poc ficant hores de provar com van algunes coses mirant de referència la plantilla que havia trobat i alguns vídeos de YouTube aconseguí acabar amb el meu projecte anomenat “Abstractos”.



Així és com ha quedat el meu joc després de moltes hores de treball i recerca, però per a arribar fins aquí hem de tornar a l'inici.

Tot va començar amb la primera versió anomenada MARK I, i sí, va ser inspiració de les pel·lícules de Iron Man. En aquesta primera versió vaig centrar-me a programar els primers prototips de sprites que com he explicat anteriorment són la part visual.

Vaig començar pel fons, els personatges i monstres. Aquesta primera versió la veritat que no era per a res estèticament atractiu, això és perquè se'm dona fatal dibuixar i perquè volia tenir algun tipus de sprite semblant al que necessitaria per poder començar a programar el funcionament del joc. Els sprites que vaig fer originalment són aquests:

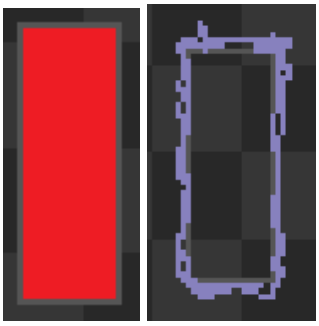


Ja havia creat un fons, un heroi i el llop o el monstre. Després d'haver-los creat els vaig assignar a un objecte dins del meu programa per a poder posar-me a escriure el codi fent d'aquesta manera funcionar el joc de la manera adequada.
Llavors el primer que havia de fer és posar d'alguna manera que el llop, que en aquest cas era el monstre que havia de matar, tingués una certa vida establerta, per a aconseguir això, en l'objecte del llop vaig crear un nou event anomenat “CREATE” aquest event bàsicament el que fa és que en iniciar-se el programa o crear-se aquest objecte dins del joc executa el seu contingut una vegada, llavors tenint aquesta informació vaig escriure el següent codi

```
max_HP=55;
hitpoints = max_HP;
```

aquestes dues serien les variables principals del llop. Ara mateix possiblement t'estaràs preguntant el que és una variable, per a explicar-ho d'una manera molt simple, imaginem que hi ha una habitació, al qual li poses un nom, per exemple “max_HP”, aquesta habitació podrà emmagatzemar el que tu vulguis sigui un número un text, etc.., el que té de bo les variables és que pots anar canviant el número o text emmagatzemat dins d'aquestes habitacions cada vegada que tu vulguis durant l'execució del programa.
Ara que ja sabem que és una variable i hem declarat 2 variables hem de pensar una manera d'aconseguir una barra de vida que vagi disminuint mentre jo vaig clicant al

llop, el primer que vaig fer va ser fer 2 sprites més 1 per a la barra de vida i un altre que sigui com el marc d'aquesta barra de vida,



En tenir aquests 2 sprites vaig descobrir una manera de poder fer el que volia mitjançant un event anomenat “DRAW”, amb aquest event el que puc fer és vincular el sprite del llop amb el de la vida i d'aquesta manera aconseguir que un interactui amb l'altre,per a aconseguir dibuixar aquesta barra de vida vaig haver d'escriure aquest codi,

```
draw_sprite(HP_BOX, 0, 400, 400);
var _fill = hitpoints / max_HP;
draw_sprite_ext(HP_FILL, 0, 400,400, 1, _fill, 1, c_white, 1.0);
```

Aquesta línia de codi a l'inici dibuixa el marc de la barra de vida a unes coordenades del joc determinat en aquest cas x=400,i=400, aquests valors són la quantitat de píxels de separació que hi ha en cada eix de coordenades, després en la segona línia declaro una variable anomenada “_fill” aquesta variable bàsicament ha de tenir números entre el 0 i l'1, per tant, el que he fet és agafar la vida del monstre i dividir-lo entre el mateix per a aconseguir el 1, t'estaràs preguntant per què necessites que el número sigui entre el 0 i el 1, perquè això és perquè en dibuixar el sprite de la barra de vida en la tercera línia necessito l'escala a la qual vull dibuixar aquesta imatge, per tant, el que jo vaig pensar és el següent que quan li dones clic al llop la vida anirà baixant, per tant, el valor de _fill que estic usant per a indicar al fet que escala vull dibuixar la barra de vida ja no serà 1 sinó 0.9 o 0.8 així fins a arribar al 0 això el que permet fer és que cada vegada que la variable de _fill disminueixi el sprite de la barra de vida sigui dibuixada a una escala més petita, per tant, fent l'efecte que la vida va baixant.

Per a aconseguir que el joc registri cada vegada que el llop està clicat i només el llop usi un event anomenat ”LEFT PRESSED”, aquest event permet al joc registrar quan aquest objecte en concret ha estat pressionat amb el clic esquerre, llavors ara que tenim l'event havia de pensar com fer disminuir la variable _fill en detectar un clic, llavors per a fer això vaig escriure el següent codi,

```
with(Upgrade_Damage)

var _damage=Damage;
hit_monster(_damage);
```

aquí tinc una variable que es diu Damage que és la quantitat de mal que hauria d'infligir sobre el llop aquesta variable anteriorment té una línia de codi ”with(Upgrade_Damage)” això és perquè la variable Damage ho tenia assignat en un altre objecte, per tant, havia de cridar-lo des de l'objecte del llop usant la funció de with, usant aquesta variable s’executa una funció feta específicament per a fer el càlcul necessari per a disminuir, la funció es veia així,

```
function hit_monster(_damage){
var _total_damage = min(Llop.hitpoints,_damage);
Llop.hitpoints -= _total_damage;
if (Llop.hitpoints <= 0) {
instance_destroy(Llop);
}
}
```

aquí a part de fer disminuir el número de _fill també controlava una altra cosa, usant aquesta línia de codi,

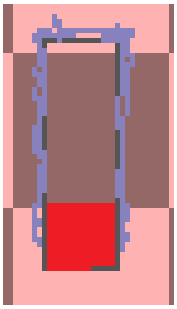
“

```
if (Llop.hitpoints <= 0) {
instance_destroy(Llop);
}
```

Aquí comprovo si la vida del llop és menor o igual a zero, això ho faig perquè vull que a la qual arribi la vida a 0 el sprite de llop desaparegui, indicant que està mort. Després de tot l'anterior vaig aconseguir fer funcionar la barra de vida.

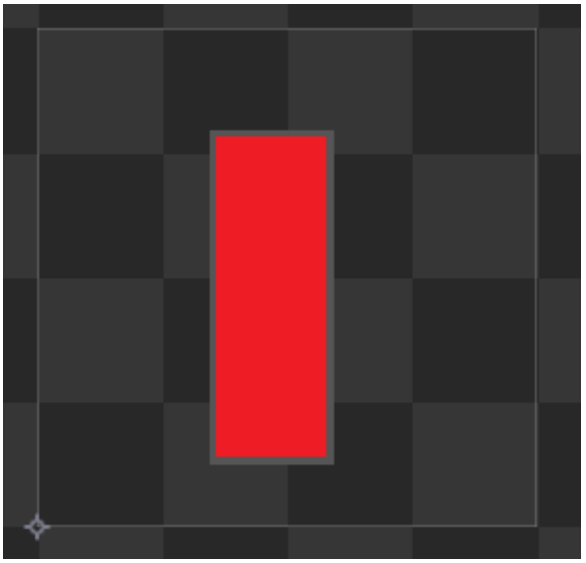


Encara que em sentia molt feliç perquè es veia bastant bé, aquesta felicitat es va acabar ràpidament quan vaig anar a clicar diverses vegades al llop i la barra de vida feia això.

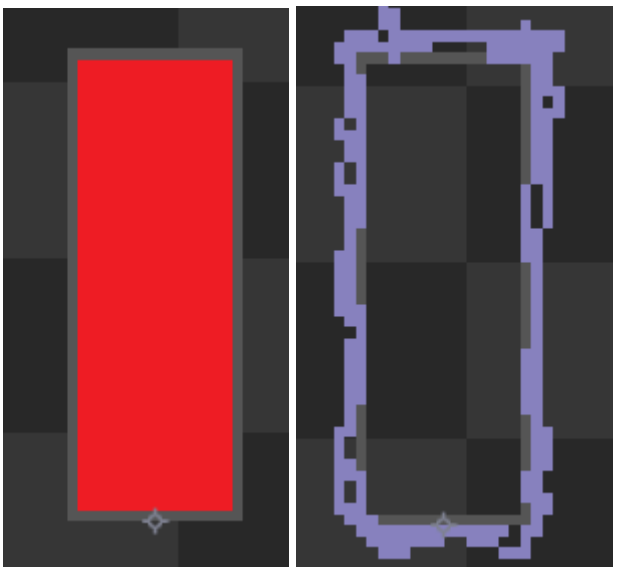


Per si no ho has pogut veure la barra de vida se'n sortia fora del seu marc, com a bon programador que soc vaig estar 5 hores fent de tot per a veure si alguna cosa ho arreglava, al final després de molt de temps vaig acabar trobant l'error que estava cometent.

Els sprites tenen un “centre d'imatge” o dita d'una altra manera “punt de referència”, el que passava és que jo havia posat aquests centres a baix a l'esquerra de l'objecte

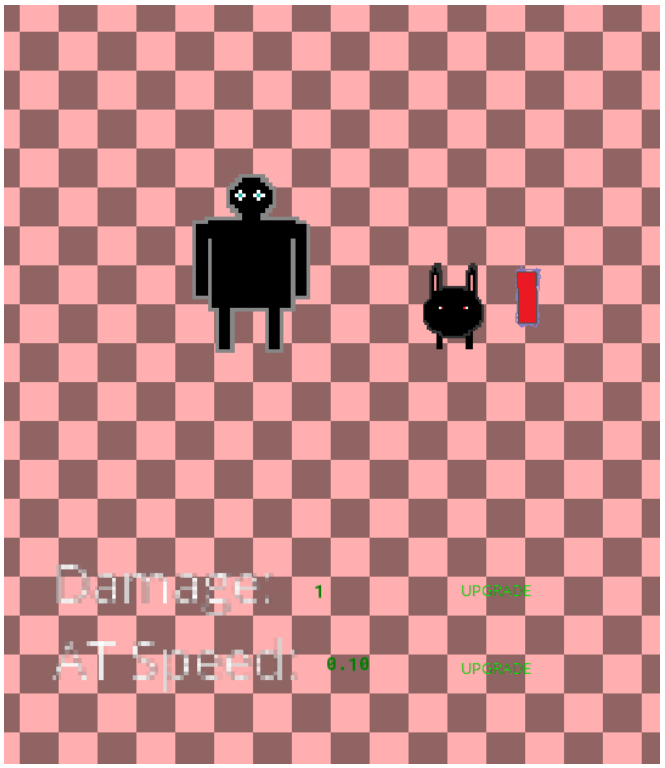


Quan vaig veure això vaig anar provant i després de provar diverses vegades esbrini que havia d'haver-ho posat en la part inferior de la barra quedant així:



En fer aquest canvi ja tenia una base del joc on si vas clicant al llop va baixant la seva vida i et va indicant quanta vida té mitjançant la barra.

Després de fer això vaig decidir afegir en una part de la pantalla com un quadre d'informació on diu el mal que fas i la velocitat d'atac, la velocitat d'atac de moment no ho estaria usant, però ho he creat ja perquè tenia plans d'usar-ho més endavant, aquest apartat d'informació va quedar així:



Això va ser com a vaig acabar el MARK I perquè no acabava de trobar una manera de fer que quan jo faci clic en el botó de upgrade Damage augmentarà el número mostrat en la part inferior i augmentarà també quan de vida li trec al monstre.

MARK II.

En aquesta versió del joc va haver-hi moltes millores, per a aconseguir l'èxit vaig haver d'acudir a la meva tutora de TR que intento ajudar-me en algunes coses, però per falta de coneixement del Game Maker no ha pogut ajudar-me a solucionar molts problemes, per la qual cosa per suggeriment d'ella i per a la nostra salut mental, vaig haver d'usar nostres volgudes IA, jo li preguntava maneres que es podien fer algunes coses i ell em donava suggeriments o possibles causes d'errors dins del programa, amb aquest suport i unes tardes de no saber que estava malament amb el programa (perquè les IA a vegades m'embolicaven encara més), els presento el MARK II:



Com ja podràs veure en aquesta versió hi havia ja molts canvis, el primer que veuràs és que ja els personatges són una mica més bonics, el quadre d'informació es veu bastant millor i és molt més fàcil de llegir el que posa, també va haver-hi alguns canvis del programa en l'àmbit funcional, ara quan fas clic al monstre et van donant monedes a dalt a la dreta de la pantalla que després pots gastar a millorar els teus atacs això inclou la força i la velocitat d'atac.

El canvi més gran en aquesta versió comparat amb l'altre és que ara l'heroi també va atacant al monstre cada cert temps, per a aconseguir això he hagut de pensar molt i consultar diverses pàgines web, unes altres de les coses que he hagut de fer van ser les animacions dels personatges usant els frames per a donar-los una mica més de vida, després d'anar provant diverses coses he acabat amb el següent codi: En l'event de “CREATE” de l'objecte de l'heroi

```
image_speed=0.8;
hit_timer=2
hit=false
ATK_Speed_upgrade=100;
alarm[0] = ATK_Speed_upgrade;
_gold=1;
var gold = _gold;
```

Aquí declaro varias coses, per exemple image_speed=0.8; per a determinar la velocitat a la que va l’animació del personatge.

```
ATK_Speed_upgrade=100;
alarm[0] = ATK_Speed_upgrade;
```

En aquestes 2 línies de codi el que aconsegueixo fer és que el ATK_Speed_upgrade sigui igual a 100 i que alarm[0] = ATK_Speed_upgrade, això el que fa és que després de la creació de l'objecte de l'heroi que és a l'inici del joc després de 100 segons l'event de l'alarma 0 serà executat una vegada, explicaré la funció d'aquest event més endavant.

```
_gold=1;
var gold = _gold;
```

Aquí declaro una variable que és la de l'or per a poder més tard usar-la.
M'agradaria abans de seguir amb l'explicació de l'heroi, explicar com funciona el programa de: “Upgrade_AT_Speed”, “Upgrade_Damage”.

Aquests 2 objectes són els que pots veure en la part inferior del joc, són els botons de upgrade, en pressionar un d'aquests botons, hauria d'augmentar el número que es pot observar a l'esquerra al mateix temps ha de fer efecte a l'heroi i als clics que fa l'usuari per a matar el monstre.

Per a fer això he hagut de fer 5 events, un event de CREATE, un de STEP un de DRAW un de LEFT PRESSED i un altre de LEFT RELEASED.
En l'event de Create de l'objecte “Upgrade_Damage” he declarat les següents variables

```
_price=5;
Damage=1;
pressed = false;
se_puede=false
no_puede=true
```

En aquestes variables declarem el preu, la força i altres que estaré modificant en altres events.
En l'event de DRAW tinc el següent codi:

```
draw_self()
draw_set_color(c_green);
draw_text(200, 611,Damage);
draw_text(420, 625,_price);
```

Amb això aconsegueixo dibuixar els botons en el seu lloc i al mateix temps aconsegueixo posar el preu de la millora i la força dels atacs.

Després tenim l'event LEFT PRESSED en aquest event simplement tinc una variable

pressed = true;

que en ser pressionat la variable es converteix en true.

En l'event LEFT RELEASED tinc el seguent codi:

```
if (pressed) {
        if(O_HERO1._gold>=_price){
        audio_play_sound(yesss, 0, false);
        Damage++;
        O_HERO1._gold=O_HERO1._gold-_price
        _price++
        pressed = false;
        }
}
```

Amb això aconsegueixo que quan la variable de "pressed" estigui activada i tinguem igual o més diners que el necessari per a comprar la millora executa les següents coses

```
audio_play_sound(yesss, 0, false);
        Damage++;
        O_HERO1._gold=O_HERO1._gold-_price
        _price++
        pressed = false;
```

Amb el comando d'àudio_play_sound el que amb si és posar un so cada vegada que fas la compra de la millora, aquest àudio en aquest cas és un àudio meu dient yes, després aconseguim augmentar una vegada la variable de "Damage",

```
O_HERO1._gold=O_HERO1._gold-_price
_price++
```

Amb aquest codi restem el preu de la millora als diners que tenim en aquest moment i també augmentem per un el preu de la millora.
Al final de tot tornem a posar la variable “pressed” en “false” perquè es pari la funciona fins a tornar a ser pressionat.

Després tenim l'event de “STEP” amb aquest codi:
se_puede=(O_HERO1._gold>=Upgrade_Damage._price)
no_puede=(O_HERO1._gold<Upgrade_Damage._price)

image_index = ((se_puede || !no_puede) ? 0 : 1);

això ho he fet per a poder canviar de color al botó de upgrade depenent si tens diners per a comprar o no.

Per a aconseguir això he hagut de fer 2 “frames” que podríem dir que són 2 fotos del mateix objecte en estats distints, amb l'única diferència sent entre un i l'altre, que un és verda i l'altre és vermell.

se_puede=(O_HERO1._gold>=Upgrade_Damage._price)
no_puede=(O_HERO1._gold<Upgrade_Damage._price)

Aquí tenim 2 variables que detecten quan tenim mes diners que el preu de la millor o menys.

image_index = ((se_puede || !no_puede) ? 0 : 1);

Amb això aconsegueixo anar canviant entre la lletra vermella i verda depenent de quin de les variables està activa.

L'objecte “Upgrade_AT_Speed” té els mateixos events de “LEFT PRESSED” i “STEP”
però per exemple en l'event de “CREATE”declarem les següents variables:
_price=5;
ATK_Speed=1

```
interval=0.6
se_puede=false;
no_puede=false;
```

Aquí declaro el preu de la millora i variables que modificaré en altres events de l'objecte.

En l'event de “DRAW”tinc el següent:

```
draw_self();
draw_set_color(c_green);
draw_text( 200, 685,ATK_Speed);
draw_text(420, 690,_price);
```

És bastant semblant a la de l'objecte que he explicat anteriorment, però aquest dibuixa el preu de la millora i la velocitat d'atac.

En l'event de “LEFT REALEASED”he escrit aquest codi:

```
if (pressed) {
        if(O_HERO1._gold>=_price){
        audio_play_sound(yesss, 0, false);
        O_HERO1._gold=O_HERO1._gold-_price
        _price++
   O_HERO1.ATK_Speed_upgrade= O_HERO1.ATK_Speed_upgrade*interval
        ATK_Speed++
        }
}
```

Aquí faig la mateixa comprovació de si, el botó està pressionat i si tinc més diners que el necessari per a fer la compra.

Si aquestes 2 condicions es compleixen
reprodueix el so de yess gravat per mi anteriorment.

```
O_HERO1._gold=O_HERO1._gold-_price
_price++
```

Amb aquestes línies de codi tornem a restar-nos els diners de la millora i augmentem la quantitat necessària per un,

```
O_HERO1.ATK_Speed_upgrade= O_HERO1.ATK_Speed_upgrade*interval
ATK_Speed++
```

Aquí tenim canvi de 2 variables, tenim primer que augmentem la velocitat d'atac de l'heroi multiplicant la “ATK_Speed_upgrade” per la variable “interval” que és igual a 0,6. Amb això aconseguim fer disminuir el número un 40%, el segon canvi és augmentar la variable “ATK_Speed” per 1 per a poder canviar el número que es mostra en l'apartat inferior del joc.

Ara que ja sabem com van aquests 2 objectes, seguim amb el programa de l'heroi.
En l'event “ALARM[0]”tinc el següent codi:

```
alarm[0] = ATK_Speed_upgrade;
hit=true;
hit_timer = 1
_gold=_gold+Upgrade_Damage.Damage0.2
```

en aquest event creem un bucle amb un temporitzador usant la variable “ATK_Speed_upgrade” establint cada vegada que s'activi la”ALARM[0]” quina hit=true i hit_timer = 1,al mateix temps fem augmentar la quantitat de monedes, però d'aquesta manera només ens dona el 20% que ens estaria donant normalment fent clic

En l'event de “STEP”he escrit les següents línies de codi:

```
var _damage=Upgrade_Damage.Damage;
if (hit==true){
hit_timer -= 1
hit_monster(_damage);
}
```

```
if (hit_timer <= 0) {
        hit = false;
                }

var _monstruo = instance_nearest(411, 406, mainMALOS);
if (instance_exists(_monstruo) && (_monstruo.pressed || hit)) {
    sprite_index = ataque;
    image_speed = 1.5;
}
else if (sprite_index == ataque && image_index >= image_number - 1) {
    sprite_index = HERO1;
    image_speed = 0.8;
}
```

en primer lloc, tenim això:

```
if (hit==true){
hit_timer -= 1
hit_monster(_damage);
}
if (hit_timer <= 0) {
        hit = false;
                }
```

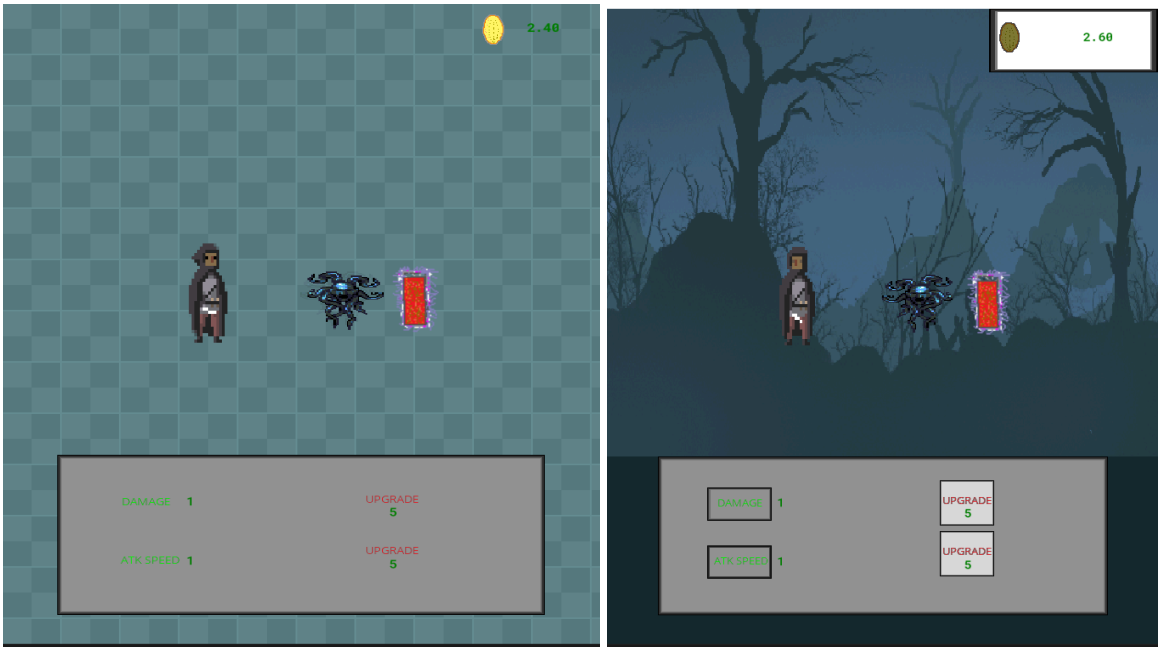
Amb això comprovem que l'heroi hauria d'atacar i executem la funció “hi_monster”, aquesta és la mateixa funció que usem en la del monstre per a disminuir la vida.

En segon lloc:

```
var _monstruo = instance_nearest(411, 406, mainMALOS);
if (instance_exists(_monstruo) && (_monstruo.pressed || hit)) {
    sprite_index = ataque;
    image_speed = 1.5;
}
else if (sprite_index == ataque && image_index >= image_number - 1) {
    sprite_index = HERO1;
    image_speed = 0.8;
}
```

Encara que hi ha moltes coses escrites, amb això simplement faig reproduir l'animació d'atac de l'heroi.
Amb això havia ja acabat el meu MARK II

MARK III, última versió



Entre aquestes 2 versions majorment la diferència serà en els “SPRITES” és a dir en la part visual, tenim un fons més bonic, he afegit un fons als diners i els botons de millora per a poder ser més fàcil llegir-ho, però també tenim un canvi molt gran que va ser afegir més monstres.
Quan em vaig proposar de fer aquest canvi, em vaig adonar d'un greu error que havia comès a l'hora de fer el meu joc, des del principi, el que estava fent és que declarava moltes variables en cada objecte per separat, quan el que havia de fer és declarar-lo en una i usar-la com el pare d'altres objectes modificant aquestes variables en comptes de tornar a declarar-les, llavors després d'haver-me donat compte d'aquest error i haver-lo arreglat he pogut posar-me a pensar com poder ficar més monstres.
Per a fer això he fet un altre objecte anomenat “OBJ_SPAWN” aquest objecte no tenia cap mena d'aspecte visual simplement era un objecte invisible que usava per a 2 coses, prenent les seves coordenades per a saber on apareixen els monstres i per a fer la llista de monstres posant-los per ordre de dificultat.

En l'event “CREATE” tinc declarat aquestes 2 variables:

```
stage=0
alarm[0] = 1;
```

En el evento “ALARM[0]”tengo lo siguiente:

```
var _monster = OBJ_MAGO;
        switch (stage) {
case 0:
        _monster = OBJ_medusa;
        break;
case 1:
        _monster = OBJ_MAGO;
        break;
case 2:
        _monster = obj_verdecito;
        break;
case 3:
        room_goto(win);
        exit;

}
instance_create_layer(411,401, "mobs", _monster);
```

Aquí tinc una llista dels 3 monstres que es podran matar en aquest joc ordenant-los per dificultat.

Per últim, en l'event “STEP” tinc aquest codi:

```
with (mainMALOS) {
var _monster = instance_nearest(411, 406, mainMALOS);
if(_monster.hitpoints<=0)
{
        OBJ_SPAWN.stage += 1;
   OBJ_SPAWN.alarm[0] = 1;
instance_destroy(_monster);
}
}
```

Aquest codi va comprovant sempre si el monstre que està en pantalla en aquest moment està mort, dit d'una altra manera si la seva vida és inferior o igual a 0, si està mort destrueix el monstre actual i fa executar l'“ALARM[0]” per a fer aparèixer al següent en la llista de monstres.

12. Conclusions

Cal destacar que, durant el procés de fer la meva part pràctica, he tingut molts problemes a l'hora d'escriure el codi, moltes vegades preguntant a les IA per a veure si podrien ajudar-me a trobar les fallades l'única cosa que aconseguia és embolicar-me encara més, per tant, he après 2 coses molt importants, tenir molta paciència perquè les coses acaben sortint i ser més autònom buscant-me maneres de solucionar els meus problemes.

Podria dir que he confirmat la meva hipòtesi: he creat un videojoc amb nocions de dos llenguatges previs. Aquests dos llenguatges m'han ajudat a entendre el nou llenguatge de programació que per mi era totalment desconegut i també m'ha ajudat a entendre l'estructura de la programació del videojoc.

11. Webgrafia

https://es.wikipedia.org/wiki/ENIAC
https://es.wikipedia.org/wiki/John_Mauchly
https://es.wikipedia.org/wiki/J._Presper_Eckert
https://es.wikipedia.org/wiki/Crisis_del_videojuego_de_1983
https://es.wikipedia.org/wiki/Unity_(motor_de_videojuegos)
https://es.wikipedia.org/wiki/GameMaker_Studio
https://es.wikipedia.org/wiki/Amazon_Lumberyard
https://es.wikipedia.org/wiki/C_(lenguaje_de_programaci%C3%B3n)
https://es.wikipedia.org/wiki/C%2B%2B
https://es.wikipedia.org/wiki/C_Sharp
https://es.wikipedia.org/wiki/Java_(lenguaje_de_programaci%C3%B3n)
https://es.wikipedia.org/wiki/JavaScript
https://es.wikipedia.org/wiki/Python
https://es.wikipedia.org/wiki/Lua_(lenguaje_de_programaci%C3%B3n)
https://www.lifewire.com/the-eniac-legacy-8749699
https://penntoday.upenn.edu/news/worlds-first-general-purpose-computer-turns-75
https://www.britannica.com/biography/John-Mauchly
https://www.britannica.com/biography/J-Presper-Eckert-Jr
https://www.britannica.com/technology/ENIAC
https://www.computerhistory.org/timeline/1952/
https://computerhistory.org/profile/steve-russell/